PUBLIC VERSION - SANITIZED FOR DISCLOSURE

Specific technical values including hostnames, IP addresses, file paths, and attack payloads have been redacted to protect the production system. All findings, descriptions, impacts, and remediation steps are fully intact.

SECURITY ASSESSMENT REPORT

reservexbd.com

Public / Sanitized Edition

Field	Details
Target	reservexbd.com
Assessment Date	March 18, 2026
Assessor	Sagar Biswas
Methodology	Passive and semi-active recon - DNS enumeration, HTTP probing, surface analysis, technology fingerprinting, authentication flow review, infrastructure mapping
Scope	reservexbd.com and all discovered subdomains, media buckets, and linked endpoints
Classification	**PUBLIC - Sanitized for General Disclosure**
Report Version	1.0
Total Findings	15 findings across 5 severity levels
Sanitization Note	Specific technical identifiers redacted. All findings, impacts, and fixes retained in full.

Redaction Notice

What Has Been Redacted

- Specific origin server hostname and IP addresses
- Cloud storage bucket URLs and exposed file path examples
- Exact attack payload URLs and exploit bypass strings
- Internal URL slugs and path structures that could aid targeting

What Remains Fully Intact

- All 15 finding IDs, titles, severities, and CVSS scores
- All CVSS vectors and CWE references
- Full descriptions of every vulnerability
- All evidence categories and non-sensitive evidence values
- Complete impact assessments for every finding
- All remediation steps in full detail
- Attack chain analysis (with specific URLs replaced by descriptions)
- Positive findings and disclaimer

Redacted values are shown as: **[REDACTED]**

1. Executive Summary

This report covers the results of a security assessment conducted on reservexbd.com on March 18, 2026. The assessment was passive and semi-active, meaning no actual exploits were run against the live system. All findings are based on observable DNS records, HTTP responses, page content, and technology fingerprinting.

A total of 15 security findings were identified. Two are rated Critical and need to be fixed immediately. Four more are rated High and should be fixed within the next few days. The remaining findings range from Medium to Informational.

Risk Summary - All 15 Findings	
Severity Level	**Number of Findings**
CRITICAL	2
HIGH	4
MEDIUM	5
LOW	3
INFORMATIONAL	1

Two Things That Need to Be Fixed Today

RXBD-001 - Origin Server Hostname Exposed

The www DNS CNAME record leaks the real hosting backend hostname. Anyone who finds this can bypass the WAF, rate limiting, and DDoS protection entirely. Fix by whitelisting only Cloudflare IP ranges in the hosting firewall settings.

RXBD-002 - Publicly Accessible Cloud Storage Bucket

The media storage bucket is publicly accessible and all file paths are embedded in the page HTML. Anyone can enumerate and download everything stored there. Fix by setting the bucket to private and serving media via a Cloudflare-proxied custom subdomain.

Also Fix Immediately (Business-Critical)

RXBD-007 - Broken Payment Gateway

The bKash and Nagad payment iframe is failing to load on restaurant pages. Users cannot pay right now. This is a revenue-stopping bug likely caused by a CSP misconfiguration.

RXBD-004 + RXBD-014 - OTP Brute Force Risk

OTP brute force on signup and password reset with no visible rate limiting means any account can potentially be taken over. This applies to both the signup flow and the password reset flow.

2. Attack Chain Analysis

This section explains the most dangerous multi-step attack scenarios that are possible if the critical and high-severity issues are not fixed. Specific URLs and hostnames have been replaced with descriptions.

Primary Chain - Account Takeover and Booking Manipulation

Risk Level: CRITICAL | Probability: HIGH - all component vulnerabilities confirmed or strongly indicated

1. Step 1: Attacker discovers the real origin server hostname via a simple public DNS CNAME lookup (RXBD-001)
2. Step 2: Sends all attack traffic directly to the origin server, bypassing Cloudflare WAF and all rate limiting controls

3. Step 3: Enumerates valid user phone numbers by observing different error messages on login and password reset pages (RXBD-005, RXBD-014)
4. Step 4: Brute-forces the OTP on the password reset endpoint for a target account since there is no rate limiting in place (RXBD-004)
5. Step 5: Successfully resets the victim's password and gains full account access
6. Step 6: Accesses the victim's bookings, cancels reservations, and causes forfeit of deposit payments (RXBD-008)
7. Step 7: Views the victim's personal data including phone number, email, restaurant preferences, and special occasion notes

Secondary Chain - Phishing via Open Redirect

Risk Level: HIGH | Probability: MEDIUM-HIGH

8. Step 1: Attacker crafts a malicious login URL using the ?next= open redirect vulnerability (RXBD-003)
9. Step 2: Shares the link via social media, SMS, or email to ReserveX users - the link looks legitimate because it uses the real domain
10. Step 3: User clicks the link and sees the real ReserveX login page, enters their credentials
11. Step 4: After a successful login, the user is silently redirected to the attacker's fake site
12. Step 5: Attacker now has valid credentials for the victim's ReserveX account

3. Infrastructure Summary

General technology stack and infrastructure details observed during the assessment. Specific hostnames, IP addresses, and internal identifiers are redacted in this public version.

Infrastructure and Technology Stack	
Primary Domain	reservexbd.com
Resolved IPs	[REDACTED]
CDN / WAF	Cloudflare (proxied)
Nameservers	Cloudflare nameservers
Mail Servers	Cloudflare mail routing (MX)
Hosting Platform	Cloud PaaS (App Platform)
Origin Hostname (LEAKED)	[REDACTED]
Media Storage	Object/blob storage service - Singapore region
Subdomain Found	www subdomain CNAME record exposes real origin backend hostname - see RXBD-001
Zone Transfer Attempted	FAILED (FORMERR) - Cloudflare correctly blocks AXFR zone transfers
Backend Framework	Django (Python) - inferred from URL patterns, ?next= redirect param, and URL slug conventions
Frontend	Server-rendered HTML with JavaScript
Payment Gateways	bKash and Nagad
SMS OTP Provider	Third-party SMS gateway (vendor not identified)
Email Obfuscation	Cloudflare Email Address Obfuscation via cdn-cgi/l/email-protection (XOR encoding)

4. Detailed Findings

All 15 findings in full detail, sorted from Critical down to Informational. Each includes the CVSS score, CVSS vector, category, CWE reference, description, evidence, impact, and remediation steps. Evidence items marked [REDACTED] contain specific technical values disclosed only to the authorized team.

RXBD-001 Origin Server Hostname Exposed - Cloudflare Bypass Possible

Severity	**CRITICAL (CVSS Score: 9.1)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L`
Category	Infrastructure / Network
CWE Reference	CWE-200 - Exposure of Sensitive Information

Description

The DNS CNAME record for the www subdomain directly reveals the real origin hosting platform hostname. An attacker can send HTTP requests directly to this origin, completely bypassing Cloudflares WAF, DDoS protection, rate limiting, and IP-based geo/bot blocking. All security controls provided by Cloudflare become ineffective once the real origin is known.

Evidence

DNS Record: [REDACTED - Disclosed to Authorized Team Only]
Origin Resolution: [REDACTED - Disclosed to Authorized Team Only]
Attack Vector: `Attacker sets Host header to the main domain and sends requests directly to the discovered origin hostname - Cloudflare is bypassed entirely`

Impact

Complete bypass of Cloudflare WAF, DDoS protection, and rate limiting. All other attack vectors in this report become significantly easier to exploit because none of the Cloudflare security controls apply.

How to Fix It

Priority: Immediate

13. In your App Platform settings, enable Trusted Sources Only and configure the app to only accept traffic from Cloudflare IP ranges.
14. Add a firewall rule to whitelist only Cloudflares published IP ranges (https://www.cloudflare.com/ips/).
15. Configure Djangos ALLOWED_HOSTS so that direct requests to the origin hostname return a 400 or 403 error.
16. Do not expose the origin subdomain in any public DNS records. Use a private CNAME or configure the app to only serve traffic routed via Cloudflare.

RXBD-002 Publicly Accessible Cloud Storage Media Bucket

Severity	**CRITICAL (CVSS Score: 9.3)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
Category	Cloud Storage / Misconfiguration
CWE Reference	CWE-284 - Improper Access Control

Description

All media files are served from a publicly accessible cloud storage bucket. The bucket name and full file paths are embedded in every pages HTML source. If the bucket ACL is set to public-read, anyone can enumerate all contents, download all restaurant photos, and discover other file paths such as user profile pictures, uploaded documents, or backup files. If the write ACL is also misconfigured, an attacker could overwrite files.

Evidence

Bucket URL: [REDACTED - Disclosed to Authorized Team Only]
Example Public File Path 1: [REDACTED - Disclosed to Authorized Team Only]
Example Public File Path 2: [REDACTED - Disclosed to Authorized Team Only]
Path Pattern Leaked: `Year/month-based directory structure is fully visible in page HTML`
Cache Directory: `Internal thumbnail/image processing cache directory is publicly accessible - reveals internal system structure`

Impact

Full enumeration and download of all bucket contents. If user-uploaded content is stored here, personal data is exposed to anyone. Path traversal via predictable filenames is possible. Potential file overwrite if write ACL is open.

How to Fix It

Priority: Immediate

17. Audit your cloud storage bucket ACL and ensure it is NOT set to public. Set the bucket to private.
18. Serve media files through a Cloudflare-proxied custom domain or signed URLs with expiry times.
19. Never expose the raw storage hostname in HTML. Use a CDN subdomain proxied via Cloudflare.
20. Remove or restrict the internal /CACHE/ path as it reveals internal thumbnail generation structure.
21. Enable cloud storage access logging to audit who has been accessing files.

RXBD-003 Open Redirect via Unvalidated ?next= Parameter

Severity	**HIGH (CVSS Score: 7.4)**

CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N`
Category	Authentication / Open Redirect
CWE Reference	CWE-601 - URL Redirection to Untrusted Site (Open Redirect)

Description

The login and signup endpoints accept a ?next= query parameter to redirect users after authentication. This is a standard Django pattern, but it becomes dangerous if the value is not strictly validated to be a relative path. An attacker can craft a malicious login URL that, after the user logs in successfully, silently redirects them to an attacker-controlled website. This is commonly exploited in phishing, OAuth token theft, and credential harvesting.

Evidence

Observed Legitimate URL: `/login/?next=/dining/restaurant/[restaurant-slug]/`

Attack Payload Example: [REDACTED - Disclosed to Authorized Team Only]
Bypass Variant 1: [REDACTED - Disclosed to Authorized Team Only]
Bypass Variant 2: [REDACTED - Disclosed to Authorized Team Only]
Bypass Variant 3: [REDACTED - Disclosed to Authorized Team Only]

Note: `Django's built-in url_has_allowed_host_and_scheme() should prevent this IF properly called with an explicit allowed_hosts parameter - verify in code`

Impact

Phishing via trusted domain URL. OAuth and session token leakage. User credential redirection. Very high credibility attacks because the URL legitimately starts with the real domain.

How to Fix It

Priority: High

22. Make sure Django's url_has_allowed_host_and_scheme() is called on every ?next= value before redirecting.
23. Pass allowed_hosts={'yourdomain.com'} explicitly and do not rely on defaults.
24. Reject any ?next= value that is not a relative path starting with /.
25. Add server-side validation: if the next value contains :// or starts with //, reject it immediately.

RXBD-004 OTP Brute Force - No Confirmed Rate Limiting on Phone Verification

Severity	**HIGH (CVSS Score: 8.1)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
Category	Authentication / Brute Force
CWE Reference	CWE-307 - Improper Restriction of Excessive Authentication Attempts

Description

The signup and password reset flows use a numeric OTP delivered via SMS. Typical OTPs are 4 to 6 digits, meaning only 10,000 to 1,000,000 possible combinations. If no server-side rate limiting, lockout, or CAPTCHA is enforced on the OTP verification endpoint, an attacker can brute-force all possible values for any target phone number. The Resend OTP button also creates an SMS flooding vector against any phone number.

Evidence

Signup Page: `/signup/ - OTP form visible with Verify and Resend buttons, no CAPTCHA present`

Password Reset Page: `/password-reset/ - sends OTP to phone for password reset, no CAPTCHA present`

Risk Factor 1: `No CAPTCHA visible on OTP entry forms`

Risk Factor 2: `No lockout mechanism confirmed from UI inspection`

Risk Factor 3: `Resend OTP button present with no visible cooldown timer`

Risk Factor 4: `OTP is numeric only - typically 4 to 6 digits, which is a very small search space`

Impact

Full account takeover of any registered user. Mass account registration with fake phone numbers. SMS flooding and abuse of any mobile number.

How to Fix It

Priority: High

26. Implement server-side rate limiting: max 3 to 5 OTP attempts per session, then lock the account for 15 minutes.
27. Make OTPs expire after no more than 5 minutes.
28. Add rate limiting on the Send OTP endpoint: max 3 SMS per phone number per hour.
29. Use a 6-digit OTP as a minimum. Consider time-based OTP (TOTP) with shorter validity windows.
30. Log and send alerts when there are repeated OTP failures for the same phone number.
31. Add a CAPTCHA before OTP send for new or suspicious sessions.

RXBD-005 User Account Enumeration via Login Error Messages

Severity	**HIGH (CVSS Score: 7.5)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
Category	Authentication / Information Disclosure
CWE Reference	CWE-204 - Observable Response Discrepancy

Description

The login form accepts either phone number or email address. If the application returns different error messages for account not found versus wrong password, an attacker can figure out which phone numbers or email addresses have registered accounts. Since mobile numbers follow a

predictable format, the search space is limited. Combined with the OTP brute-force finding, a full account takeover chain becomes very realistic.

Evidence

Login URL: `/login/ - accepts Phone Number or Email`

Enumeration Vector 1: `Differing error messages between unknown phone and wrong password responses leak account existence`

Enumeration Vector 2: `Response time differences (timing oracle) between known and unknown accounts`

Enumeration Vector 3: `Bangladesh mobile number format is predictable - reduces brute-force search space significantly`

Impact

An attacker can build a list of all registered user phone numbers and emails, which can then be used for targeted phishing, SMS spam, or credential stuffing attacks.

How to Fix It

Priority: High

32. Return a generic error message for all login failures: 'Invalid credentials' - never distinguish between unknown user and wrong password.
33. Make sure response time is consistent regardless of whether the account exists. Add an artificial delay if needed.
34. Implement account lockout after 5 to 10 failed attempts with exponential backoff.
35. Add a CAPTCHA after 3 consecutive failures from the same IP address.

RXBD-006 Missing Critical HTTP Security Headers

Severity	**HIGH (CVSS Score: 7.2)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N`
Category	Security Headers / Configuration
CWE Reference	CWE-693 - Protection Mechanism Failure

Description

The application is missing or has weak implementations of critical HTTP security headers. The payment iframe failure observed in the browser is likely linked to a missing or broken Content-Security-Policy. Without these headers, the app is exposed to a range of common web attacks that are otherwise easy to prevent. The cross-origin media loading from cloud storage also suggests that CORS and CSP policies have not been fully configured.

Evidence

Payment iframe Error: `Observed on a restaurant page: 'Could not load payment page. The payment service refused the connection or timed out.'`

Cross-Origin Media: `All images loaded from cloud storage URL cross-origin without clear CORS/CSP policy defined`

Missing Header 1: `Content-Security-Policy`
Missing Header 2: `X-Frame-Options`
Missing Header 3: `X-Content-Type-Options`
Missing Header 4: `Referrer-Policy`
Missing Header 5: `Permissions-Policy`
Missing Header 6: `Strict-Transport-Security (HSTS)`
Note: `Cloudflare may inject some headers automatically, but application-level headers should not rely solely on CDN injection`

Impact

Without CSP: XSS attacks can execute arbitrary JavaScript. Without X-Frame-Options: clickjacking on login and payment pages is possible. Without HSTS: SSL stripping attacks can downgrade connections. Without X-Content-Type-Options: MIME-sniffing attacks become viable.

How to Fix It

Priority: High

36. Add to Django settings: SECURE_BROWSER_XSS_FILTER = True, SECURE_CONTENT_TYPE_NOSNIFF = True, X_FRAME_OPTIONS = DENY.
37. Set SECURE_HSTS_SECONDS = 31536000 and SECURE_HSTS_INCLUDE_SUBDOMAINS = True in Django.
38. Implement a Content-Security-Policy header that explicitly whitelists your own domain, the media bucket domain, and the payment gateway domains.
39. Set Referrer-Policy to strict-origin-when-cross-origin.
40. Fix the broken payment iframe by ensuring the payment provider domain is in the CSP frame-src directive.
41. Use https://securityheaders.com to scan and verify headers after implementation.

RXBD-007 Broken Payment Gateway Integration

Severity	**MEDIUM (CVSS Score: 6.5)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:H`
Category	Business Logic / Payment
CWE Reference	CWE-400 - Uncontrolled Resource Consumption / Payment Integrity

Description

On restaurant detail pages, the payment modal shows an error indicating the payment iframe failed to load. The bKash and Nagad payment integration is completely non-functional. This could be due to incorrect CSP directives blocking the payment provider domain, a misconfigured iframe URL, wrong API credentials in production, or a CORS issue. This is a business-critical failure as users cannot complete payments.

Evidence

Affected Page: `/dining/restaurant/[restaurant-slug]/`

Observed Error: `'Could not load payment page. The payment service refused the connection or timed out.'`

Payment Methods Listed: `bKash and Nagad`

Impact Type: `Business logic failure - core transactional feature is completely non-functional`

Impact

Users cannot pay deposits right now. The business is losing revenue. If partial payment states are being recorded without completion, there may also be a way to bypass payment verification entirely.

How to Fix It

Priority: High - fix immediately as this is business-critical

42. Open browser DevTools on the restaurant page and check the Console and Network tabs for the exact error when the payment modal opens.
43. Verify that bKash and Nagad API credentials are correctly configured in the production environment.
44. Check the CSP frame-src header and make sure the payment gateway domain is explicitly whitelisted.
45. Test the payment flow end-to-end in a staging environment before deploying the fix.
46. Add server-side payment status verification and never trust client-side payment confirmation alone.

RXBD-008 Potential IDOR on Booking and Reservation Objects

Severity	**MEDIUM (CVSS Score: 6.8)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`
Category	Authorization / IDOR
CWE Reference	CWE-639 - Authorization Bypass Through User-Controlled Key

Description

The platform lets users view, cancel, and manage their own bookings through My Bookings. Reservation platforms very commonly use sequential or predictable booking IDs. If the application does not verify that the requesting user actually owns the booking, any logged-in user can view, cancel, or modify someone else's reservation just by changing the ID number in the URL. This is a classic Insecure Direct Object Reference (IDOR) vulnerability.

Evidence

Booking Management: `/help-center/ - 'Go to My Bookings on your profile section'`

Authentication Required: `Yes - users must log in to access bookings`

Risk Indicator: `Reservation platform with deposit payments - bookings likely have numeric or UUID identifiers`

Test Needed: `Authenticated test with two separate accounts required to fully verify object-level authorization - was not performed in this passive assessment`

Impact

Unauthorized viewing of other users' personal data including name, phone, restaurant, date, and special requests. Unauthorized cancellation of reservations. Deposit theft if cancellation triggers a refund to the attacker's account.

How to Fix It

Priority: High

47. On every booking-related endpoint, verify that request.user matches booking.user before returning any data.
48. Use Django's get_object_or_404(Booking, pk=booking_id, user=request.user) pattern throughout.
49. Use UUIDs instead of sequential integers for booking IDs to prevent enumeration.
50. Implement Django REST Framework's IsOwner permission class for all booking API endpoints.
51. Add automated tests that verify user A cannot access user B's bookings.

RXBD-009 Contact Form - Potential CSRF and Spam Abuse

Severity	**MEDIUM (CVSS Score: 5.4)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:L`
Category	CSRF / Abuse
CWE Reference	CWE-352 - Cross-Site Request Forgery

Description

The contact form collects sensitive personal information including name, email, phone, subject, booking number, and message. Without a CSRF token and rate limiting, attackers can forge requests from authenticated sessions or flood the support team with thousands of fake messages. No CAPTCHA was detected on this page.

Evidence

Contact Form URL: `/contact/`

Fields Present: `First Name, Last Name, Email Address, Phone Number, Subject, Order/Booking Number, Message`

No Visible CAPTCHA: `No CAPTCHA element detected in the page HTML`

Note: `Django includes CSRF protection by default via CsrfViewMiddleware - needs verification that it has not been disabled`

Impact

Support team gets flooded with spam and fake messages. CSRF could force authenticated users to submit contact messages impersonating them. Booking numbers collected via autofill could be leaked.

How to Fix It

Priority: Medium

52. Verify that Django's CsrfViewMiddleware is enabled and that the csrf_token tag is present in the form template.
53. Add Google reCAPTCHA v3 or hCaptcha to the contact form.
54. Implement rate limiting: max 3 contact form submissions per IP per hour.
55. Do not echo back user-supplied input in email notifications without proper sanitization.

RXBD-010 Django Admin Panel Exposed at Default Path

Severity	**MEDIUM (CVSS Score: 5.9)**
CVSS Vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`
Category	Configuration / Admin Exposure
CWE Reference	CWE-530 - Exposure of Backup File to Unauthorized Control Sphere

Description

Django applications expose the admin panel at /admin/ by default. While this was not directly accessible during the assessment, the default path is widely known and automated bots probe it constantly. If the Cloudflare bypass from RXBD-001 is used, the admin panel may be reachable directly. A successful brute-force or credential stuffing attack against admin credentials would be devastating.

Evidence

Default Django Admin Path: `/admin/ (standard well-known path)`

URL Pattern Evidence: `URL structure confirms Django framework (accounts/signup/, ?next= param, URL slug conventions)`

Accessibility: `Could not be directly fetched during assessment - likely blocked by Cloudflare WAF or returns 404`

Risk: `Default path is well-known and probed by bots constantly. Cloudflare blocking does not prevent direct-to-origin attacks (see RXBD-001)`

Impact

If admin credentials are ever compromised, the attacker gets full database access, can view all user personal data, manipulate restaurant information, and potentially achieve remote code execution via the Django shell.

How to Fix It

Priority: Medium

56. Move the admin panel to a non-default, hard-to-guess path in urls.py.

57. Restrict admin access by IP allowlist using Cloudflare Access or a custom Django middleware.
58. Enable two-factor authentication on all Django admin accounts using django-otp or similar.
59. Never reuse the admin password for any other service.
60. Store the admin URL in an environment variable rather than hardcoding it in urls.py.

RXBD-011 No robots.txt File - Search Engine Exposure of Internal Paths

Severity	**LOW (CVSS Score: 3.1)**
CVSS Vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`
Category	Information Disclosure
CWE Reference	CWE-200 - Exposure of Sensitive Information

Description

The website does not appear to serve a robots.txt file. Without it, search engines crawl every publicly reachable URL including sensitive paths. Attackers can use Google dork queries to discover these paths from search engine caches without ever touching the server directly.

Evidence

robots.txt Accessible: `No - request returned a permissions error`
Crawlable Sensitive Path 1: `/admin/`
Crawlable Sensitive Path 2: `/accounts/`
Crawlable Sensitive Path 3: `/api/`
Crawlable Sensitive Path 4: `/password-reset/`
Crawlable Sensitive Path 5: `/dining/surprise-me/`

Impact

Search engines index sensitive URLs. Attackers can use Google dork queries to discover hidden paths without performing any active scanning against the server.

How to Fix It

Priority: Low

61. Create a /robots.txt file that disallows: /admin/, /accounts/, /api/, and /password-reset/.
62. Add a sitemap.xml reference in robots.txt for SEO benefit.
63. Use X-Robots-Tag headers on sensitive authenticated pages to prevent search engine indexing.

RXBD-012 Email Address Disclosure via Cloudflare Obfuscation Bypass

Severity	**LOW (CVSS Score: 3.7)**
CVSS Vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N`
Category	Information Disclosure
CWE Reference	CWE-200 - Exposure of Sensitive Information

Description

The site uses Cloudflare Email Address Obfuscation to hide email addresses from scrapers. However, this is simply XOR encoding and any developer or automated tool can decode it in seconds by inspecting the HTML source. Team email addresses for support, general info, and business partners are all embedded in the HTML in this decodable format.

Evidence

Protected Email 1: `Support email address - encoded via cdn-cgi/l/email-protection in the footer`

Protected Email 2: `Partners email address - encoded via cdn-cgi/l/email-protection in /partner-with-us/`

Protected Email 3: `Business email address - encoded via cdn-cgi/l/email-protection in /contact/`

Decode Method: `Cloudflare email obfuscation is XOR with a key - freely decodable by any web scraper or developer in seconds`

Impact

Team email addresses can be harvested for spam, phishing, or spear phishing attacks specifically targeting the team.

How to Fix It

Priority: Low

64. Use a contact form instead of displaying email addresses directly wherever possible.
65. For email addresses that must be shown, consider image-based rendering or CSS direction tricks as extra obfuscation.
66. Monitor the support inbox for unusual spam volume as an indicator that addresses have been harvested.

RXBD-013 Missing sitemap.xml - SEO and Crawl Surface Management

Severity	**LOW (CVSS Score: 2)**
CVSS Vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:N`
Category	Informational / SEO
CWE Reference	N/A

Description

No sitemap.xml was found. While this is not a direct security issue, the absence means search engines have no authoritative map of the site and will crawl based on discovered links. This can result in indexing pages that were not intended to be prominent in search results.

Evidence

sitemap.xml Accessible: `No - file not found`

Impact

Primarily an SEO and crawl management concern. Low security impact, but worth addressing as part of general web hygiene.

How to Fix It

Priority: Informational

67. Use Django's built-in sitemap framework to automatically generate /sitemap.xml.
68. Exclude sensitive and admin paths from the sitemap.
69. Reference the sitemap URL in robots.txt.

RXBD-014 No Rate Limiting on Password Reset - Phone Number Enumeration via SMS

Severity	**MEDIUM (CVSS Score: 5.3)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:L`
Category	Authentication / Enumeration
CWE Reference	CWE-307 - Improper Restriction of Excessive Authentication Attempts

Description

The password reset page accepts a phone number and sends an OTP. If the response is different for registered versus unregistered phone numbers, attackers can figure out who has an account. Without rate limiting the same phone number can be sent OTPs over and over again, constituting an SMS flood attack that abuses the SMS gateway and costs the company money while harassing the target phone owner.

Evidence

Password Reset URL: `/password-reset/`

Mechanism: `Enter phone number - receive OTP - reset password`

Enumeration Vector: `Differential response between registered and unregistered phone numbers allows account discovery`

SMS Flood Vector: `No visible CAPTCHA or rate limit control on the OTP send button`

Impact

User phone number enumeration. SMS cost abuse. Harassment of arbitrary phone numbers. This is commonly used as the first step in a targeted account takeover attack.

How to Fix It

Priority: Medium

70. Return identical responses whether or not the phone number is registered in the system.
71. Limit OTP sends to max 1 per phone number per 2 minutes and max 5 per day.
72. Add IP-level rate limiting on the reset endpoint.
73. Add a CAPTCHA to the password reset form.

RXBD-015 Unverified Email on Account Registration - Trust Escalation Risk

Severity	**MEDIUM (CVSS Score: 5)**
CVSS Vector	`CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N`
Category	Authentication / Business Logic
CWE Reference	CWE-345 - Insufficient Verification of Data Authenticity

Description

Registration requires phone verification via OTP, but email is described as optional ('If Added and Verified' per the help center). Booking confirmations are sent to both SMS and email. If email is accepted without verification, an attacker can register with someone else's email address, causing that person to receive booking confirmation emails they did not authorize. This also enables impersonation in support interactions.

Evidence

Source: `/help-center/ - 'Email: Detailed confirmation with restaurant information (If Added and Verified)'`

Registration Flow: `Phone is verified via OTP. Email verification status is conditional and not required.`

Risk: `If email is accepted without verification, users can register with any arbitrary email address they do not own`

Impact

Email spoofing within the platform. Victims receive unwanted booking confirmation emails they never signed up for. Impersonation in support interactions using the victim email address.

How to Fix It

Priority: Medium

74. Require email verification via a confirmation link before sending any emails to that address.
75. Do not send booking confirmations to an email address until it has been verified.

76. Use Django's email verification flow (django-allauth or a custom implementation) for this purpose.

5. Remediation Priority Order

All 15 findings sorted by recommended fix priority. The effort column is a rough estimate of developer time. Fix the Critical and High items first.

#	ID	Finding	Effort	Impact
1	RXBD-001	Fix origin hostname exposure - enforce Cloudflare-only access	Low	Critical
2	RXBD-002	Restrict cloud storage bucket to private access	Low	Critical
3	RXBD-007	Fix broken payment gateway integration	Medium	Business-Critical
4	RXBD-004	Implement OTP rate limiting and lockout	Medium	High
5	RXBD-003	Validate ?next= parameter to prevent open redirect	Low	High
6	RXBD-005	Standardize login error messages to prevent enumeration	Low	High
7	RXBD-006	Add security headers (CSP, HSTS, X-Frame-Options)	Medium	High
8	RXBD-008	Implement object-level authorization on booking endpoints	Medium	High
9	RXBD-010	Move Django admin to non-default path, add IP restriction and 2FA	Low	High
10	RXBD-014	Rate limit password reset SMS endpoint	Low	Medium
11	RXBD-009	Add CSRF verification and CAPTCHA to contact form	Low	Medium
12	RXBD-015	Require email verification before sending confirmations	Medium	Medium
13	RXBD-011	Create robots.txt disallowing sensitive paths	Low	Low
14	RXBD-012	Review email address exposure strategy	Low	Low
15	RXBD-013	Add sitemap.xml for SEO and crawl management	Low	Informational

6. What Is Already Working Well

Not everything needs to be fixed. The following positive observations were noted during the assessment and should be maintained.

- Cloudflare DNS protection is in place and AXFR zone transfer attempts are correctly blocked.
- HTTPS is enforced across the entire site and no plain HTTP is being served.
- Phone OTP verification is required for registration, which prevents mass fake account creation.
- bKash and Nagad payment integrations use Bangladesh Bank-authorized gateways.
- Email addresses are obfuscated in HTML via Cloudflare Email Obfuscation (even though the encoding is weak, it is better than nothing).
- Cloudflare is used as a CDN and WAF, providing baseline DDoS protection for the site.
- Privacy Policy, Terms of Service, and Refund Policy are all present and accessible to users.

7. Disclaimer

This assessment was conducted with authorization from the system owner. All testing was passive and semi-active. No exploitation was attempted at any point during the assessment. Findings are based entirely on observable DNS records, HTTP responses, page content analysis, and technology fingerprinting.

Authentication-gated functionality including My Bookings and payment flows was not tested with authenticated access. A full penetration test with authenticated access is strongly recommended to fully validate the IDOR, payment bypass, and API security findings noted in this report.

This is the sanitized public version of this report. Specific technical values including hostnames, IP addresses, file paths, and attack payloads have been removed. The full private version with all technical details has been shared only with the authorized team.